SFWRENG 3S03

Complete Lecture Notes

Matthew Farah

April 4, 2026

Contents

1 Introduction 3

2 Faults 4

3 Types of Testing 5

4 Testing Levels 7

5 Test Suites 8

6 Code Coverage 9

7 SKIPPED 11

8 Relations 12

9 Equivalence Classes 13

10 Midterm Details 14

11 SKIPPED 15

12 Examples of Equivalence Partitions 16

13 MC/DC, Structural Testing, and The V Model 18

14 Code Reviews 20

15 Verification 21

16 SKIPPED 22

17 SKIPPED 23

18 Formal Reasoning 24

19 Formal Modelling 26

20 Petri-nets and CTL 28

21 CTL Query Examples 30

22 Petri-net Theory 31

23 An example of Petri-net Analysis 32

24 Software Quality **33**

25 Factors in Testing **34**

26 Testing Plans **35**

27 System Plans, Unit Plans, and Root Cause Analyses **36**

Lecture 1

Introduction

- Code shouldn't ever be superfluous.
  - Superfluous code increases the risk of faults and bugs existing in the codebase.
- Good tests are engineered, automated, and risk-focused.

Lecture 2

Faults

- Testing is defined as the process of evaluating software to estimate how well it meets some set of criteria.
  - One of the primary goals of testing is to identify 'faults' in a program.
- Faults in programs are defined as anything which may cause the program to behave in an unintended way during execution.
  - Synonymous with 'bug' or 'defect'.
- 'Errors' and 'failures' are defined as specific kinds of faults.
  - A program is said to error if it enters a bad state during its execution.
  - A failure is an observed instance of the program deviating from expected behaviour during execution.
- Tests can only be used to prove or disprove the existence of faults in code.
- A test case is a specific choice of inputs with a given set of initial conditions designed to verify if the program works as intended.
  - Test cases use 'test oracles' to determine this.
    - Test oracles compare the actual and expected result of the program in a test case.
- Edge case inputs are synonymous with boundary values.
- Tests should be designed systematically, using domain knowledge about the program's inputs.

Lecture 3

Types of Testing

- 'Input cases' are groups of inputs to a program which cause it to behave similarly during execution.
  - An 'edge case' is an input belonging to either multiple input cases or none at all.
    - Testing edge cases is key to ensuring programs behave as intended.
  - When input cases are well defined, a single representative test case from a group can be used to adequately verify that the program works as intended for all inputs in that test case's group.
- Tests should ideally be written and execute as quickly as possible.
  - However, testing may be constrained by its cost and the availability of code artifacts to test on.
- The kinds of bugs being tested for dictate the kinds of tests to be used.
- Code 'linting' is the act of enforcing certain standards and code style throughout a codebase.
- Certain kinds of tests require the program to be executed to work, while others do not.
  - This splits tests into two distinct categories.
- All tests are either:
  1. Dynamic tests.
     - Tests programs by executing them.
     - Used to find runtime or integration bugs.
     - Should be down throughout the Continuous Integration / Continuous Development (CI/CD) process.
  2. Static tests.
     - Tests programs <u>without</u> executing them.
     - Used to catch bugs early and perform code linting.
     - Should be performed prior to any dynamic tests.
- As well, certain kinds of tests work using or not using available knowledge about the code or its structure.
  - This splits tests into another three distinct (somewhat more ambiguous) categories.
- All tests are either:
  1. Black-box tests.
     - Tests programs without using <u>any</u> knowledge of the code or its structure.
     - Must be done as a dynamic test.
     - Performed when:
       - No knowledge of the code or its structure is available.
       - The test is verifying that any changes made to the code or its stucture shouldn't affect its behaviour.
  2. Grey-box tests.
     - Tests programs by using <u>some</u> knowledge of the code and/or its structure.

    - Typically done as a dynamic test.
    - Performed when:
        * The test is verifying that units are properly integrated.
3. White-box tests.
    - Tests programs by using <u>full</u> knowledge of the code and its structure.
    - Can be done as either a dynamic or static test.
    - Performed when:
        * The test must do anything not possible by black-box or grey-box testing.

- Requirements define characteristics a given system must abide by.
- Requirements are divided into two distinct categories:
    1. Functional requirements.
        - Describe what the system must do.
            * For example, the features or behaviour it must have.
    2. Non-functional requirements.
        - A system's non-functional requirements describe the performance needs the system must satisfy.
            * For example, the speed it must operate at.
- Tests can be written to verify that a system is compliant with its functional or non-functional requirements.
    - This once again places tests into two distinct categories.
- All tests are either:
    1. Functional tests.
    2. Non-functional tests.

Lecture 4

Testing Levels

- A test's 'testing level' describe the scope of the software being tested.
- The three testing levels used in the course are:
  1. Unit.
     - Tests small, meaningful pieces of the code.
     - Always done as white-box tests.
     - Generally inexpensive to develop.
     - Should be run as often as possible.
  2. Integration.
     - Tests how units work together.
     - Done by treating units are black-boxes.
     - Low to moderately expense to develop.
     - Should be done after the creation of every new unit.
  3. System.
     - Tests the entire system.
     - May be done as white or black box tests.
     - Important functional and non-functional tests are typically done at this level.
     - Most expensive testing level.
     - Should be done prior to every release.
- There are three primary approaches to designing good integration tests.
  1. The big-bang approach:
     - Create integration tests for all units at once.
     - TLDR; shit, dumb, cringe.
  2. The top-down approach:
     - Create integration tests starting with the largest units/components first.
     - TLDR; Mid asf.
  3. The bottom-up approach:
     - Create integration tests starting with the smallest units/components first.
     - TLDR; based, epic.
- System tests are designed to do one of two things:
  1. Verify the system.
     - Ensure that the system meets agreed upon specifications.
  2. Validate the system.
     - Ensure that the system delivers on what its user(s) want.
     - Validating system tests are also called acceptance tests.
- Tests can be developed manually or by an automated process.
  - Context and the kind of test being developed dictate the method by which it should be developed.

Lecture 5

Test Suites

- A test suite is a collection of tests for a program/system.
  - 'SUT' = 'System Under Test'.
- There exist two kinds of unit tests:
  1. Solitary.
     - Isolates unit tests by using 'test doubles' to simulate its dependencies.
  2. Social.
     - Any unit test that isn't solitary.
- There are five important kinds of test doubles:
  1. Dummies.
     - Uses empty objects to simulate what dependencies return to the unit.
  2. Fakes.
     - Uses minimally functioning objects to simulate what dependencies return to the unit.
  3. Stubs.
     - Uses a static, pre-determined values to simulate what dependencies return to the unit.
  4. Spies.
     - Stubs that additionally record what the unit does with the value returned to it.
  5. Mocks
     - A minimal implementation of the dependency the unit relies on.
- The type of test double to be used depends on both the unit itself and the bugs being tested for.
- Automated testing helps ensure system-breaking dependency changes are identified.
- Testing should always be done as early and often as possible.
  - This minimizes the cost and time required to identify and fix bugs.

Lecture 6

Code Coverage

- Bugs tend to cluster together in systems.
  - Generally, 20% of code is responsible for 80% of bugs.
- Tests can be 'formal' or 'informal'.
  - Formal tests are done in a specific way and check specific things.
  - Informal tests look at subjective things without a rigorous methodology.
- Some examples of formal and informal tests are as follows:
  - Exploratory testing.
    - TLDR; Manually fucking around with the system and seeing what happens.
    - Informal.
  - Fuzz testing.
    - TLDR; Automated exploratory testing.
    - Informal.
  - Model-based testing.
    - TLDR; Creating formal models of the system to automatically generate tests.
    - Formal.
  - Specifications-based testing.
    - TLDR; Using invariants and pre/post-conditions of the system to test its behaviour.
    - Formal.
- All tests are either:
  1. Robust.
     - Checks <u>both</u> valid and invalid inputs.
  2. Normal.
     - Checks <u>only</u> valid inputs.
- As well, all tests are either:
  - Strong.
    - Makes no assumption about how many bugs the code can have.
  - Weak.
    - Assumes the code being tested can only have one bug in it.
- Every test can be described as some combination of Robust/Normal and Strong/Weak.
- A program's Control Flow Graph (CFG) is a graph representing the sequence of instructions that can be executed by the program.
- A path is a sequence of valid execution steps in a CFG.
  - Paths must start at a valid entry point of the program and end at some exit point.

- A set of paths are said to be linearly independent if none of them can be expressed as some combination of the other paths in the set.
- A test suite's coverage is a measurement (metric) describing how much of the code is being tested.
  - Coverage can be defined in multiple ways.
- The five main definitions of test coverage are:
  1. Statement/line coverage.
     - Achieved when every line of code is tested.
  2. Condition coverage.
     - Achieved when every boolean of the code has been tested as true and false.
  3. Branch/Decision coverage.
     - Achieved when every possible branch in the code has been tested.
  4. Modified Condition/Decision Coverage (MC/DC).
     - Achieved when:
       (a) Every entry and exit point has been invoked.
       (b) Every decision outcome has been tested.
       (c) Each condition in a decision that can independently change the outcome of the decision has been tested.
  5. Path coverage.
     - Achieved when all possible linearly independent paths through the CFG have been tested.

Lecture 7

SKIPPED

- SKIPPED

Lecture 8

Relations

- Fault injection involves deliberately introducing faults into the system.
  - Done to evaluate efficacy of existing tests.
    - Kinda like coverage metrics.
  - More expensive than coverage metrics.
- Fault injection can be done in one of two ways:
  1. Compile-time fault injection.
     - Introduces faults into the source code.
     - Typically done manually.
     - Performed in three possible ways:
       (a) Operand replacement.
         - Ex. Changing $a + b$ to $a + c$.
       (b) Expression modification.
         - Ex. Changing $a + b$ to $a - b$.
       (c) Statement modification.
         - Ex. Deleting $x := 1$ from code.
  2. Runtime fault injection.
     - Introduces faults as the program is running.
     - Typically automated.
     - Typically performed by use of special software triggers.
- A binary relation is a set R representing ordered associations between some sets A and B.
  - $R \subseteq A \times B$.
  - Elements are expressed as $(a, b) \in R$.
    - $a \in A, b \in B$.
- Equivalence relations are special kinds of binary relations.
- Equivalence relations $R \subseteq A \times A$ must satisfy the following three properties:
  1. Reflexivity.
     - $\forall a \in A; (a, a) \in R$.
  2. Symmetry.
     - $\forall a_1, a_2 \in A; (a_1, a_2) \in R \implies (a_2, a_1) \in R$.
  3. Transitivity.
     - $\forall a_1, a_2, a_3 \in A; (a_1, a_2) \in R \wedge (a_2, a_3) \in R \implies (a_1, a_3) \in R$.
- Binary and equivalence relations are fundamental to test design theory.

Lecture 9

Equivalence Classes

- A system's domain is the set of all its possible inputs.
- An equivalence class is an equivalence relation for some subset of the program's domain ($E \subseteq D$).
  - For $x \in D$, E_x is an equivalence class for x if $\forall y \in D; x \equiv y \implies y \in E_x$.
- If two inputs belong to the same equivalence class, the program behaves the same for both inputs.
  - Useful to reduce the number of tests needed.
- A system's domain can be 'partitioned' into multiple equivalence classes.
- $P = \{ E_{x_1}, E_{x_2}, \ldots, E_{x_n} \}$ is a partition of D if its equivalence classes:
  1. Are disjoint.
     - $\forall i \neq j; E_{x_i} \cap E_{x_j} = \emptyset$.
  2. Equal the domain.
     - $\bigcup_{i=1}^{n} E_{x_i} = D$.
- If the domain can be partitioned, only n-many tests (the number of equivalence classes) are needed to test the system.
- In practice, very few program domains can be partitioned into meaningful equivalence classes.
  - To partition most domains, an equivalence tolerance must be introduced.
    - Ex. x is equivalent to y if $|x - y| \leq 1$.
      - Here, our tolerance is 1.
      - Tolerance is also called delta.
- Partitions with tolerance no longer guarantee that two inputs in the same equivalence class make the system act the same on those inputs.
  - Since non-trivial tolerances violate transitivity.
  - The smaller the tolerance, the less significant this becomes.
- A partition P_1 finer than partition P_2 if $P_2 \subseteq P_1$
  - Occurs when P_1 has a smaller tolerance than P_2.
  - P_2 would be called a courser partition.
- A test suite for some partition is called adequate if every equivalence class has a test for it.
  - If a test suit is adequate for a partition, it is adequate for courser partitions.

Lecture 10

Midterm Details

Besides the following hints about the midterm, the rest of the lecture was just a recap about test doubles.

- Key midterm details:
  - Will include questions on:
    - Partitions.
    - Testing techniques.
  - Will have multiple choice and short answer questions.
  - Will have at least one question about a proof.
    - Probably about showing if a set is an equivalence class or not.

Lecture 11

SKIPPED

- SKIPPED

Lecture 12

Examples of Equivalence Partitions

- Test coverage is often best determined using a system's control flow graph.
- The number of inputs to a program does not necessarily determine the number of equivalence partitions that need to be made.

Example 1: Say we wanted to test the behaviour of the following program:

Algorithm 1 The program we wish to test

```
procedure PROGRAM(x, y, z)
    if x ≤ 0 then
        return −1
    end if
    if z ≤ 0 then
        return 0
    end if
    if x ≥ 10 then
        r ≔ x · 2
    else
        r ≔ x
    end if
    if y ≠ 0 then
        r ≔ r − y
    end if
    return r
end procedure
```

Assume x, y, and z can take on the value of any real number. It would be impossible actually to try and test all possible values of x, y, and z to verify the program works as intended. How many tests would it take to adeque

(a) How could we break down the program into as few equivalence classes as possible verify its behaviour?

Solution:

To break down the program into equivalence classes, we need to consider each variable independently and see (by inspection) if the program behaves differently (if at all) when that variable changes.

(I) Construct equivalence classes for x.

For x, we can see that the program may returns different things when:

- $x \leq 0$.
- $x \geq 10$.

This also means that when $0 < x < 10$, the program also behaves differently (since some other return statement is fired). Therefore, we could break x into the following three equivalence classes:

$$\{\, x \colon x \leq 0 \,\}, \quad \{\, x \colon 0 < x < 10 \,\}, \quad \{\, x \colon 10 \leq x \,\}$$

(II) Construct equivalence classes for y.

Likewise, we can observe that the program behaves differently when:

- $y \neq 0$.

And hence when $y = 0$ as well. So, we can construct the following equivalence classes for y:

$$\{\, y \colon y \neq 0 \,\}, \quad \{\, y \colon y = 0 \,\}$$

(III) Construct equivalence classes for z.

In the interest of not beating a dead horse, we can conclude by the exact same process that we would need the following equivalence classes to verify the behaviour of the program against z:

$$\{\, z \colon 0 \leq z \,\}, \quad \{\, z \colon z < 0 \,\}$$

This therefore gives the following set of equivalence partitions for each variable:

Variable	**Equivalence Classes**
x	$\{x : x \leq 0\}$, $\{x : 0 < x < 10\}$, $\{x : x \geq 10\}$
y	$\{y : y = 0\}$, $\{y : y \neq 0\}$
z	$\{z : z > 0\}$, $\{z : z \leq 0\}$

(b) Using the equivalence classes you constructed, how many tests are required in total to adequately test the program?

Solution:

The total number tests required for any program with n input variables $(x_1, \ldots, x_n)$ partitioned into a minimal of equivalence classes is given by:

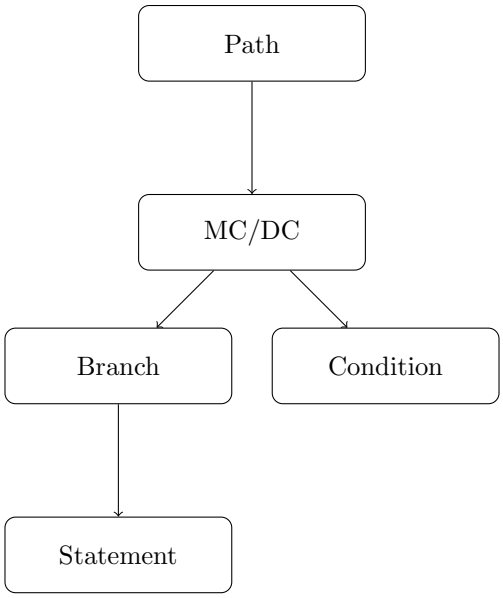
$$(\text{\# of equivalence classes for } x_1) \ldots (\text{\# of equivalence classes for } x_n)$$

So using our equivalence classes, our program would need $3 \cdot 2 \cdot 2 = 12$ tests in total.

Lecture 13

MC/DC, Structural Testing, and The V Model

- MC/DC typically requires $n + 1$ tests for a program with n inputs.
  - If not, it needs more than that.
- MC/DC works differently when testing objects and source code.
  - Source code coverage $\not\Longrightarrow$ object code coverage.
    - *Note: the prof never really defined what object code is AFAIK, but it probably just means something the program/system compiles to.*
- Structural coverage may refer to any of:
  - Statement coverage.
  - Branch coverage.
  - Condition coverage.
  - MC/DC coverage.
  - Path coverage.
- The criterion of a structural test refers to what coverage it is.
- Structural tests should be designed to consider:
  - What happens if a loop is never executed.
  - A loop is only executed once.
  - A loop is executed some arbitrary number of times.
- Structural tests should also be designed to force:
  - An immediate success.
  - An eventual success.
  - An immediate failure.
- A criterion 'subsumes' another one if satisfying it implies satisfying the other.
  - I.E. A subsumes B if satisfying A implies satisfying B.
- If a criterion subsumes another, it is considered better.
  - Doesn't mean it will necessarily encounter the same failures.
- Subsumption is transitive.
  - I.E. If A subsumes B and B subsumes C then A subsumes C.
- Structural tests criteria are ordered in terms of subsumption as follows:
  - Path coverage subsumes MC/DC coverage.
  - MC/DC coverage subsumes both branch and condition coverage.
  - Branch coverage subsumes statement coverage.



- Complexity metrics measure how difficult code is to test.
  - For example, control-flow complexity or state behaviour complexity.
- The Software Development Life-Cycle (SDLC) refers to a range of different possible development approaches teams can/should follow to work optimally.
  - An SDLC models refer to some such specified approach.
- The V-model is an SDLC model focused primarily on developing the system's requirements.
- The V-model divides up development into two main phases, done in the following order:
  1. Verification.
     - Involves insuring the system is built to do the right job.
       * Done by speaking with clients.
  2. Validation.
     - Involves insuring the system does its job right.
       * Done by checking if system meets some set of standards.

- *Aside: The difference between verification and validation is pretty easy to miss but you're supposed to think about it like, if you're trying to help someone travel, verification is about making sure that a car is good for them and validation is making sure the engine starts.*
  - *Second aside: I hate learning about project management but I cannot escape it someone save me.*

Lecture 14

Code Reviews

- Bugs identified later in the development process are:
  - More difficult to fix.
  - More expensive to fix.
- Peer review is an evaluation done by others.
  - In peer reviews, there are pros and cons associated with the level of familiarity that:
    * Your peer has with you.
    * Your peer has with the code you're working on.
    * Your peer has with the tools you're using.
- Software reviews are a subset of peer reviews.
  - There are many kinds of software reviews.
    * Software reviews range in formality.
- Good reviews should identify, besides existing bugs:
  - Potential risks.
  - Maintainability improvements.
    * *I.E. Your code sux lolmao.*
- Inspecting code front-loads the effort required to develop software.
- Fault injection can also be used to help review code to see if tests are correctly catching bugs.

Lecture 15

Verification

- Verification can be done with static analysis and/or dynamic testing.
  - *Aside: Sounds more like validation to me but idk.*
- It is unlikely dynamic testing can identify all possible faults.
  - Some faults are incredibly difficult to trigger.
- All automated software verification techniques (such as static analysis) can be categorized as:
  1. Sound.
     - Only accepts systems that don't do some particular thing.
       - I.E. No false negatives.
  2. Complete.
     - Only accepts systems that do some particular thing.
       - I.E. No false positives.
  3. Decidable.
     - Only accepts correct systems (and terminates).
- All automated verification techniques are incomplete and/or unsound and/or undecidable.
- Many important properties of a system cannot be decided.
  - So, said property of a system cannot be guaranteed after verification.
  - Many other properties, however, can be decided.
    - Or, the approximations of the property may be decidable.
- Compilers are the most common static analyzer software engineers interact with.
  - Type checking is an important, decidable property that compilers are able to verify.

Lecture 16

SKIPPED

- SKIPPED

Lecture 17

SKIPPED

- SKIPPED

Lecture 18

Formal Reasoning

- Formal reasoning is a way by which we can analyze systems mathematically.
  - Formal reasoning is necessary when approximating system behaviour is not sufficient.
    - For example, in safety-critical systems.
- Hoare logic is formal reasoning technique for programs.
- Hoare logic consists of a in a set of Hoare Triples.
  - Hoare triples consist of:
    1. A pre-condition ($\{ P \}$).
       - A pre-requisite condition of the system's variables for everything that follows.
    2. A statement (S).
       - The code being tested.
    3. A post-condition (R).
       - A state the program must be in after the statement has been executed, if the pre-condition was satisfied.
- The pre- and post-conditions are both expressed using predicate logic.
  - The pre- and post-conditions can <u>only</u> be expressed in terms of the program's inputs.
- Hoare triples are written as:
  - $\{ P \} S \{ Q \}$.
    - Can be read as "if $\{ P \}$ holds and S happens, then $\{ Q \}$ must be true."

Example 1: Find appropriate post conditions for the following set of Hoare Triples:

	$\{ P \}$	S	$\{ Q \}$
(a)	$\{ x = y \}$	$x := x + 3$	??
(b)	$\{ x > -1 \}$	$x := x * 2 + 3$	??
(c)	$\{ true \}$	$x := 5$	??

Solution:

We can reason about what the post-condition for each Hoare triple should be as follows:

(I) Find the post-condition for (a).

The pre-condition asserts that prior to executing S, x and y are equal. S just increments x by 3, and so after executing S, we can evidently see that $x = y + 3$. We can therefore make $Q = \{ x = y + 3 \}$ our post-condition.

Note here that for Hoare logic, we don't consider implementation details like integer overflow when determining our pre- and post-conditions. As well, there are multiple valid post-conditions we could have selected for this particular one. For example, $Q = \{ x > y \}$ would also be a valid post-condition. I'll talk about this a bit later when discussing weakest and strongest pre- and post-conditions later.

(II) Find the post-condition for (b).

As before, we can see that prior to doing S, x is strictly greater than -1. S multiplies x by two and adds 3 to it, and since:

$$\begin{aligned} & x > -1 \\ \implies & 2x + 3 > 2 \cdot (-1) + 3 \\ \implies & 2x + 3 > -2 + 3 \\ \implies & 2x + 3 > 1 \end{aligned}$$

And so, after assigning x to be $2x + 3$, we always have that $x > 1$. So $Q = \{ x > 1 \}$ would be a valid post-condition.

(III) Find the post-condition for (c).

The pre-condition for (c) is simply true, so no assumptions about the input x (or any other potential input variable(s) to S for that matter) are being made. However, after executing S, we at least know that x is assigned to be 5. Thus, $Q = \{ x = 5 \}$ would be a valid post-condition.

To summarize, we have that:

	$\{ P \}$	S	$\{ Q \}$
(a)	$\{ x = y \}$	$x := x + 3$	$\{ x = y + 3 \}$
(b)	$\{ x > -1 \}$	$x := x * 2 + 3$	$\{ x > 1 \}$
(c)	$\{ true \}$	$x := 5$	$\{ x = 5 \}$

- Hoare logic is often used to reason about small parts of a code base.
  - For example, methods of a system as opposed to the system itself.
- The strongest post-condition Q' is a post-condition that implies all other post-conditions.
  - If $\{ P \} S \{ Q' \}$ holds, then for any post-condition Q, $\{ P \} S \{ Q \}$ also holds.
- The weakest pre-condition P' is a pre-condition implied by all other pre-conditions.
  - If for all pre-conditions P, $\{ P \} S \{ Q \}$ holds, then $\{ P' \} S \{ Q \}$ also holds.
- Can be thought of as:
  - Q' has the strictest requirements on the program's variables after execution, so if $\{ P \} S \{ Q' \}$ holds, $\{ P \} S \{ Q \}$ must also hold since Q has less strict requirements.
  - P' makes the weakest assumptions about the program's variales, so if $\{ P' \} S \{ Q \}$ holds, $\{ P \} S \{ Q \}$ must also hold since P has stricter requirements.

Lecture 19

Formal Modelling

- Building models of system helps in the testing process.
  - Helps abstract away details unnecessary to the testing process.
  - Allows for the system to be evaluated without an actual implementation.
- Models can be:
  1. Formal.
     - Built around the system's formal requirements.
  2. Informal.
     - A rough draft of the system.
- There are many different ways to model the same system.
  - Ex. Petri-nets, state-transition systems, etc.
- There is no single 'best' model for a system.
  - The best model for a system is context-dependent.
- Model checking a system follows a two-step process:
  1. Building a formal model of the system.
     - Using the system's formal requirements.
  2. Evaluating the constructed model.
- Model checking is an undecidable problem.
  - There doesn't exist an algorithm which can always determine whether a given model of a system always behaves correctly or not.
- All languages are composed of at least two things:
  1. A concrete syntax.
     - The language's notation.
  2. An abstract syntax.
     - A set of rules governing how the language's notation can be structured.
- Formalisms are languages with semantics.
  - Semantics assign meaning to notation.
- Models are expressed using formalisms to prevent ambiguity.
- Some notable formal models include:
  - Kipke structures.
    - TLDR; Basically just finite state machines.
    - Don't think we really need to know these.

  - Timed automata.
    * TLDR; Kipke structures that additionally have:
      · An infinite state-space.
      · Timed semantics (behaviour depends on transitions and the elapsed time).
    * Also don't think we really need to know these.
  - Petri-nets.
    * TLDR; See SFWRENG-3BB4.
    * We do need to know these.

- Petri-nets are used to model:
  - Concurrency.
  - Causality.
  - Synchronization.
  - Resource constraints.

- Petri-nets verify system properties such as:
  - Reachability.
  - The existence of deadlocks.
  - Boundedness.
  - Liveliness
    * *Aside: The slides call it liveness but I'm using liveliness because liveness sounds so dumb to me.*

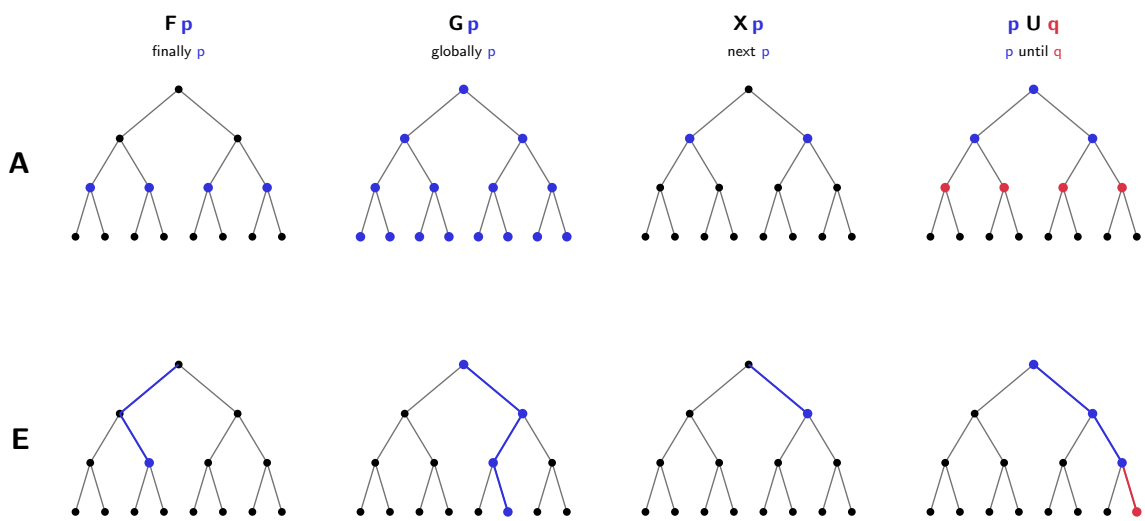
Lecture 20

Petri-nets and CTL

- Petri-nets are used to model non-determinism in software.
- The critical section of a program refers to where shared resources are stored.
  - Semaphores are used to control processes' access to the critical section.
- Inhibator arcs are arcs (arrows) that can only be taken if its input place is empty.
- Transitions may have different priorities.
  - Helps to solve conflicts and prevent deadlocks.
- The start state of a Petri-net describes its initial arrangement of tokens before any transitions are taken.
- The state of a Petri-net describes an valid arrangement of its tokens reachable from the start-state
- A path through a Petri-net is a valid sequence of its states.
- Computational Tree Language (CTL) is a query language.
  - Frequently used for evaluating properties of Petri-nets.
  - Evaluate to true or false.
- CTL statements are use two types of quantifiers:
  1. General quantifiers.
     - Quantifies over all paths from the current state.
  2. Path-specific quantifiers.
     - Quantifies along a given path from the current state.
- There are two general quantifiers in CTL:
  1. A.
     - Means "for all paths in the model".
     - *Aside: Think of it like* $\forall$.
  2. E.
     - Means "there exists a path in the mode".
     - *Aside: Think of it like* $\exists$.
- There are four path-specific quantifiers in CTL:
  1. F.
     - Means "finally".
     - *Aside: Think of it like the property should eventually hold.*
  2. G.
     - Means "globally".
     - *Aside: Think of it as meaning the property should hold for all valid states.*

3. X.
   - Means "next".
4. U.
   - Means "until".
   - *Aside: Think of it as meaning the property should hold <u>at least</u> until some other property is true (the property can still hold after the other becomes true).*

- The criteria for these quantifiers to be satisfied looks like the following when graphed, where:
  - p and q are properties of the system being queries for.
  - Blue vertices correspond to states where the query for p is true.
  - Red vertices correspond to states where the query for q is true.
  - Black vertices correspond to states where the query may or not be true (doesn't matter which).



- A marking vector v describes the state of a given Petri-net.
  - An index i of v represents a place in the Petri-net.
  - $v[i]$ represents the number of tokens in place i in the given state.

Lecture 21

CTL Query Examples

- The incidence map of a Petri-net describes how its tokens can flow through a path.
  - Represented in matrix form.
    - Number of rows = number of places in the Petri-net.
    - Number of columns = number of transitions.
- REST OF NOTES STILL IN PROGRESS

Lecture 22

Petri-net Theory

- NOTES IN PROGRESS

Lecture 23

An example of Petri-net Analysis

- NOTES IN PROGRESS

Lecture 24

Software Quality

- Software quality of a system is a measure of its current and future risk.
- The objective of testing is to increase software quality.
  - Planning is the process of generating mechanisms to increase software quality.
    - Involves creating or using an existing testing strategy.
  - Monitoring is the process of verifying that such planning effectively increases software quality.
- It is better to create imperfect tests early than perfect tests later.
  - Acts as a form of review.
  - Encourages making incremental improvements.
- Most defects originate from requirements, not code.
  - *Aside: Lowkey cap but whatever ig.*
- All testing strategies have three major components.
  1. Focus risks.
     - What kinds of risks the testing strategy is attempting to address.
  2. Scope.
     - Which business needs the testing strategy is attempting to address.
  3. Details.
     - Specifies important considerations of the testing strategy.
     - For example:
       - How risks are identified.
       - How risks can be minimized.
       - What metrics can help track risks.
- Different testing strategies optimize software quality with respect to different constraints.
- Some examples of widely used testing strategies include:
  - The cleanroom strategy.
    - Places an emphasis on the identification of risks.
  - The Software Reliability in Engineering Testing (SERT) strategy.
    - Places an emphasis on creating many tests.
  - The eXtreme Programming (XP) strategy.
    - *Aside: Can't remember what she said about this one and it's not on the slides.*
    - *Aside: According to Google, emphasizes early and incremental automated testing.*

Lecture 25

Factors in Testing

- Test factors describe the quality-related attributes of a system.
  - For example:
    - Correctness.
    - Data integrity.
    - Ease of use.
      - How user-friendly the system is for the client/clients.
    - Ease of operation.
      - How easy the system is to develop, operate, maintain.
- Test factors should:
  - By decided by the client's/clients' requirements.
  - Should consider all potential risks to the system.
- The testing strategy for a system should be chosen according to:
  - The team's past project experience.
  - The team's size.
  - The team's structure.
  - The application domain of the system.
- A system's risks belong to at least one of five categories.
  1. Functional.
     - Would affect the behaviour of one or more features.
  2. Performance/scalability.
     - Self-explanitory.
  3. Security.
     - Would make the system vulnerable to something.
       - Important to note, applies to the security of the system, not of users.
     - For example:
       - Data breach risks.
       - Dependency risks.
  4. Usability.
     - Would affect the system's ease of use.
  5. Maintainability.
     - Would affect the system's ease of operation.
- The risks to a system can be described in matrix form.
  - *Aside: Will describe better and/or with examples if I have time.*

Lecture 26

Testing Plans

Aside: Didn't write many notes for this one, pretty boring stuff.

- Testing plans can be decomposed into subtasks.
  - The completion of subtasks is necessary for the completion of the plan.
- A plan's testing schedule describes the time required/order of completion for its subtasks.
  - Representable by a GANTT chart.
  - *Aside: Think of it like a directed, weighted graph.*
    - *Edges = subtasks that can be done after current subtask completion.*
    - *Weights = amount of time to complete the next subtask.*
- A path described a valid, ordered completion of subtasks in a testing schedule.
  - The critical path of a testing schedule is a path with a maximum overall duration.
- A critical dependency of a subtask A on the critical path is another subtask B also on the critical path which is scheduled immediately after A.
- A given schedule can be further decomposed into smaller subtasks.
  - Only effectively if:
    - The whole of a subtask takes more time to complete than the sum of its parts.
    - Additional resources can be allocated towards managed those further decomposed subtasks.

Lecture 27

System Plans, Unit Plans, and Root Cause Analyses

- *Aside: Root cause analysis question likely on exam.*
  - *Aside: Read M7.L4.S/UT Plan and QATeam lecture slides for more details on fish-bone diagrams and components of system/unit plans, don't wanna type it out here.*

- A testing plan is not the same as a testing schedule.
  - A given testing plan may have multiple valid testing schedules.
- A testing plan is either a:
  1. System plan.
     - Plans for testing the entire system as a whole.
  2. Unit plan.
     - Plans for testing an individual component.
     - Essentially a scaled-down system test.
- System plans are comprised of four main components.
  1. General Information.
  2. The plan.
  3. Specifications and Evaluations.
  4. Test Descriptions.
- Unit plans are also comprised of four main components.
  1. The plan.
  2. Business and structural function testing.
  3. An interface test description.
  4. Test progression.
- Key takeaways from previous testing plans can be analyzed using the 8D approach.
- A root cause analysis should be done when a bug has been caught.
  - Done to help reduce number of bugs in current/future development.
- Root cause analyses seek to answer the following five core questions regarding a:
  1. What happened?
  2. When did it happen?
     - Includes both when it was first introduced and when it was first identified.
  3. Why did it happen?
  4. How can a similar bug be prevented from being introduced in the future?
- Root cause analyses can be modelled using fish-bone diagrams.
- Root cause analysis:
  - Begins with fault selection.
  - Groups different faults according to the severity.